SIMATS ENGINEERING
SAVEETHA SCHOOL OF ENGINEERING

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code: DSA03 | Course Title: Natural Language Processing

Student Name: Ramavath Prashanth Naik

Register Number:192472272 Section/Batch: CSE-AI

1. Introduction

Morphological processing is an important part of Natural Language Processing (NLP). It studies how words are formed from roots, prefixes, and suffixes. Morphological analysis helps identify relationships between word forms and supports text normalization, search engine indexing, sentiment analysis, topic modelling, document clustering, and information retrieval.

This report addresses the five scenarios in Assessment Tool 2. Each question includes morphological analysis, a rule-based Python program, expected output, and a brief conclusion. The programs use explicit rules for the listed words and are educational examples rather than complete English morphological analyzers.

2. Question 1: Morphological Processing of Analyzing, Analysis, and Analytical

Aim

To identify roots and affixes, distinguish inflectional and derivational forms, and normalize related terms for search indexing.

Morphological analysis

Word	Root/base	Affix or structure	Type	Index representation
analyzing	analyze	-ing; restore final e	Inflectional	ANALYZE_FAMILY
analysis	analysis	Irregular lexical relationship to analyze	Derivational relationship	ANALYZE_FAMILY
analytical	analysis/analy-	-ical	Derivational	ANALYZE_FAMILY

Analyzing is a participle/gerund form of analyze. Analysis is a related noun whose form cannot be obtained reliably by simple suffix deletion. Analytical is an adjective associated with analysis. A shared family key supports retrieval, while the original word should be retained to preserve grammatical distinctions.

Python program

```
words = ["analyzing", "analysis", "analytical"]
```

```
morphology = {
   "analyzing": {
      "root": "analyze",
      "affix": "-ing (restore final e)",
      "type": "Inflectional",
      "normalized": "ANALYZE_FAMILY"
   },
   "analysis": {
      "root": "analysis",
      "affix": "Irregular lexical relationship",
      "type": "Derivational relationship",
      "normalized": "ANALYZE_FAMILY"
   },
   "analytical": {
      "root": "analysis/analy-",
      "affix": "-ical",
      "type": "Derivational",
      "normalized": "ANALYZE_FAMILY"
   }
}

for word in words:
   result = morphology[word]
   print(word, result)
```

Expected result

The program prints the original word, root/base, affix information, morphology type, and the common index key ANALYZE_FAMILY for each input.

Conclusion

A lexical-family key improves retrieval across related terms, but original word forms and part-of-speech information should be preserved.

3. Question 2: Morphological Parsing of Disagree, Agreement, and Agreeable

Aim

To detect prefixes, suffixes, base forms, derivational categories, and semantic changes.

Word	Prefix	Root	Suffix	Meaning/type
disagree	dis-	agree	None	Oppose / have a different opinion; derivational
agreement	None	agree	-ment	State or result of agreeing; derivational
agreeable	None	agree	-able	Pleasant or acceptable; derivational

The prefix dis- changes the meaning of agree. The suffix -ment forms a noun, while -able forms an adjective. These forms share a base but are not semantically interchangeable; in particular, disagree carries negative or opposing meaning.

```
words = ["disagree", "agreement", "agreeable"]

morphology = {
   "disagree": ("dis-", "agree", "None",
            "Derivational", "Have a different opinion"),
   "agreement": ("None", "agree", "-ment",
             "Derivational", "State/result of agreeing"),
   "agreeable": ("None", "agree", "-able",
             "Derivational", "Pleasant or acceptable")
```

```
}

for word in words:
    prefix, root, suffix, kind, meaning = morphology[word]
    print(word, "prefix:", prefix, "root:", root,
          "suffix:", suffix, "type:", kind,
          "meaning:", meaning, "normalized base: agree")
```

Expected result

The parser identifies agree as the shared base, records the affix for each form, and describes the semantic contribution of each derivational change.

Conclusion

Morphological features can support consistent sentiment processing, but polarity and negation must be preserved rather than discarded during normalization.

4. Question 3: Normalization of Govern, Government, and Governance

Aim

To identify derivational suffixes and levels, then group related words for topic modelling and document clustering.

Word	Root	Suffix	Word class / level	Normalized form
govern	govern	None	Verb / level 0	GOVERN_FAMILY
government	govern	-ment	Noun / level 1	GOVERN_FAMILY
governance	govern	-ance	Noun / level 1	GOVERN_FAMILY

Government is formed with -ment and governance with -ance. Both are nouns related to the verb govern, but their meanings differ: government often refers to an institution or system, while governance refers to the process or practice of governing.

```
words = ["govern", "government", "governance"]

morphology = {
    "govern": ("govern", "None", "Verb", 0),
    "government": ("govern", "-ment", "Noun", 1),
    "governance": ("govern", "-ance", "Noun", 1)
}

for word in words:
    root, affix, word_class, level = morphology[word]
    print(word, "root:", root, "affix:", affix,
          "class:", word_class, "level:", level,
          "normalized: GOVERN_FAMILY")
```

Expected result

All three words are mapped to GOVERN_FAMILY, with the original form, suffix, word class, and derivational level reported.

Conclusion

Family-level normalization can improve clustering, while retaining surface forms reduces the risk of losing useful semantic distinctions.

5. Question 4: Morphological Parsing of Activate, Activation, and Reactivation

Aim

To identify derivational layers, prefixes, suffixes, word-class changes, and a shared base representation.

Word	Prefix	Root/base	Suffix / sequence	Meaning and class
activate	None	activ-	-ate	Verb; make active
activation	None	activate	-ion/-ation	Noun; process/result of activating
reactivation	re-	activate	-ion/-ation; re- + activation	Noun; activation again

The derivational relationship can be represented as active → activate → activation → reactivation. The prefix re- adds the meaning ‘again’.

```
words = ["activate", "activation", "reactivation"]

morphology = {
   "activate": ("None", "activ-", "-ate", "Verb",
            "Make active", "activ- + -ate"),
   "activation": ("None", "activate", "-ion/-ation", "Noun",
             "Process/result of activating", "activate -> activation"),
   "reactivation": ("re-", "activate", "-ion/-ation", "Noun",
              "Activation again", "re- + activation")
}

for word in words:
   prefix, root, suffix, word_class, meaning, sequence = morphology[word]
   print(word, "prefix:", prefix, "root:", root,
       "suffix:", suffix, "class:", word_class,
       "meaning:", meaning, "sequence:", sequence,
       "normalized: ACTIVATE_FAMILY")
```

Expected result

The program records each derivational layer and maps the forms to ACTIVATE_FAMILY while retaining the prefix information.

Conclusion

A shared family representation supports indexing, and retaining re- preserves the important meaning of repetition.

6. Question 5: Inflectional Normalization of Create, Creates, and Creating

Aim

To identify inflectional suffixes and grammatical categories and normalize forms to the base verb.

Word	Suffix	Grammatical category	Normalized form
create	None	Base form	create
creates	-s	Third-person singular present	create
creating	-ing; final e dropped	Present participle / gerund	create

Creates is the third-person singular present form. Creating is a present participle or gerund form formed by dropping the final e and adding -ing. These are inflectional forms of the verb create.

```
words = ["create", "creates", "creating"]

morphology = {
   "create": ("None", "Base form", "create"),
   "creates": ("-s", "Third-person singular present", "create"),
   "creating": ("-ing (drop final e)",
            "Present participle / gerund", "create")
}

for word in words:
   suffix, category, root = morphology[word]
```

```
print(word, "suffix:", suffix, "category:", category,
    "root:", root, "normalized:", root)
```

Expected result

Each input is normalized to create, with its suffix and grammatical category recorded.

Conclusion

Inflectional normalization allows a search system to match create, creates, and creating without treating them as unrelated terms.

7. Overall Conclusion

This assessment demonstrates how morphological processing can improve search and text analytics by identifying roots, affixes, derivational relationships, and inflectional variants. The examples show that inflectional forms can often be mapped to a common lemma, while derivational forms may change word class or meaning. Therefore, normalization should not rely only on deleting suffixes. A practical NLP pipeline should preserve original words, use lexical exceptions where necessary, and retain semantic distinctions such as negation and repetition.

References / Tools

Python 3. The programs in this report use built-in Python data structures and predefined analyses for the exact words specified in Assessment Tool 2. No external dataset experiment is claimed in this report.